

F5: Mer om objekt

Malmö universitet, Institutionen för datavetenskap och medieteknik

Innehåll

- Repetition
- Enum
- JavaDoc
- Arrayer med mera
- Sammanfattning



Repetition

Repetition

- En klass är en ritning för ett objekt
- Ett objekt är en instans av en klass
- Objekt samlar variabler och metoder som relaterar till objektet
- Objektorienterad programmering gör ofta projekt mer överskådliga och lättare att bryta ner i flera mindre problem

Repetition

- Klasser har
 - Konstruktorer
 - Attribut (variabler)
 - Operationer (metoder)

Konstruktorn

- Konstruktorn ska ha samma namn som klassen.
- Alla klasser ska ha en konstruktor.
 - Glömmer du att skapa en själv så skapar kompilatorn en tom default-konstruktor.
- Klasser kan ha flera konstruktorer som överskuggar varandra.
- En konstruktor kan anropa en annan konstruktor.

Konstruktor: Exempel

TODO

Flera konstruktorer som överskuggar varandra

Konstruktorer som kallar på varandra



Enum

Enum — enumeration

- Enumeration betyder uppräknning.
- enum används för att hantera konstanter som relaterar till varandra.
- Kan användas för att slippa hantera arrayer med värden.
- Så att istället för att ha en lista över färger där 1 motsvarar *red*, 2 motsvarar *green* o.s.v. kan man skriva *red* eller *green* direkt.

Skapa en enum

```
public enum Breed {  
    GOLDENRETRIEVER,  
    BORDERCOLLIE  
}
```

Istället för *public class ClassName* skriver du *public enum EnumName* överst i filen.

Namnet måste fortfarande stämma med filnamnet.

Komma åt enums

- Du kommer åt en konstant i en enum så här:

```
Breed.GOLDENRETRIEVER
```

```
EnumName.NAMEOFCONSTANT
```

- Konstanterna i en enum är:
 - public
 - final
 - static
- Enums har inget **new** kopplade till sig

Mini-objekt

En enum kan ha en privat konstruktor och publika metoder kopplade till sig

```
public enum Breed {  
    GOLDENRETRIEVER("Golden Retriever"),  
    BORDERCOLLIE("Border Collie");  
  
    private final String breedName;  
  
    Breed(String n){ this.breedName = n;}  
    public String getBreedName() { return breedName;}  
}
```

Mini-objekt 2

```
public enum TicketType {  
    CHILD(50),  
    TEEN(75),  
    ADULT(125),  
    SENIOR(100);  
  
    private final int price;  
  
    TicketType(int price){ this.price = price;}  
  
    public int getPrice(){ return this.price;}  
}
```



JavaDoc

Dokumentera kod

- Javadoc är en särskild typ av kommentarer i Java.
- Skrivs som:

```
/**  
 * Kommentarer som beskriver koden  
 */
```

- Verktöget Javadoc kan sedan skapa en wiki-liknande hemsida utifrån kommentarerna

Vart JavaDoc skrivs

- JavaDocs används ofta ovanför
 - Klass-deklarationen
 - Metod-deklarationer
 - Instans-variabler
- Kommer efter import-satser
- Skrivs inte inuti metoder

Syntax

- JavaDoc använder taggar för att markera olika typer av information
- Några vanliga:
 - `@param`: Används för att identifiera information om parametrar till en metod
 - `@return` används för att visa vad som returneras
 - `@author` visar vem som har skrivit koden
- Massor fler

HTML

- Det är möjligt att formatera JavaDoc med HTML-taggar
- `<p>` markerar ett nytt stycke
- `<ul>` skapar punktlistor
- `<ul>`
 - `<li> punkt 1 </li>`
 - ` punkt 2 ```

Exempel

```
/**
 * Short one line description.
 * <p>
 * Longer description. If there were any, it would be
 * here.
 * <p>
 * And even more explanations to follow in consecutive
 * paragraphs separated by HTML paragraph breaks.
 *
 * @param variableName1 Description text text text.
 * @param variableName2 Description text text text.
 * @return Description text text text.
 */
public int methodName (int variableName1, double variableName2) {
    // method body with a return statement
    return 0;
}
```



Arrayer med mera

Skapa en array med objekt

- Det är ofta som vi vill ha en array med objekt av samma typ
- Vi kan exempelvis ha flera hundar per hundägare, eller flera hyllplan i en bokhylla
- Skapas på samma sätt som andra arrayer

```
Dog[] allTheDogs = new Dog[4]; // Tom array med fyra  
platser  
allTheDogs[0] = new Dog();  
allTheDogs[1] = new Dog("Goofy");  
allTheDogs[2] = new Dog("Pluto", "Mickey", Breed.MUTT);  
// allTheDogs[3] is intentionally left blank
```

Skriva ut arrayen med objekt

Det är frestande att försöka skriva ut en array på följande vis:

```
System.out.println(allTheDogs);
```

Det ger däremot en utskrift i stil med:

```
[LF5_Mer_om_objekt.Dog;@3feba861
```

Istället behöver man använda en loop

Skriva ut en array med objekt

```
for (int i = 0; i < allTheDogs.length; i++){  
    System.out.println(allTheDogs[i]);  
}
```

F5_Mer_om_objekt.Dog@5b480cf9

F5_Mer_om_objekt.Dog@6f496d9f

F5_Mer_om_objekt.Dog@723279cf

null

Det är inte heller vad vi vill se

toString-metoden

Vi har två lösningar. Antingen skriver vi något i stil med:

```
System.out.println(allTheDogs[i].getName());
```

Eller så kan vi skapa metoden *toString* i vår klass **Dog**.

```
public String toString(){  
    return this.name + " a " + this.breed;  
}
```

Ny output

```
for (int i = 0; i < allTheDogs.length; i++){  
    System.out.println(allTheDogs[i]);  
}
```

Unknown a MUTT

Goofy a MUTT

Pluto a MUTT

Lite justeringar i vår enum

```
public enum Breed {  
    GOLDENRETRIEVER("Golden Retriever"),  
    BORDERCOLLIE("Border Collie"),  
    MUTT("Mutt");  
  
    private final String breedName;  
  
    Breed(String n){ this.breedName = n; }  
  
    public String getBreedName() { return breedName; }  
}
```

Lite justeringar i vår toString

```
public String toString(){  
    return this.name+" a "+this.breed.getBreedName();  
}
```

Nytt resultat

```
for (int i = 0; i < allTheDogs.length; i++){  
    System.out.println(allTheDogs[i]);  
}
```

Unknown a Mutt

Goofy a Mutt

Pluto a Mutt

null

Enhanced for-loopar

När vi inte bryr oss om indexet i en loop kan vi skapa så kallade ”*enhanced for-loops*”:

```
for (Dog dog: allTheDogs){  
    System.out.println(dog);  
}
```

Unknown a Mutt

Goofy a Mutt

Pluto a Mutt

null

Hantera null

- null betyder att värdet är osatt
- Detta kan leda till problem när man loopar över en array
- Exempelvis om vi försöker skriva ut alla hundars ras. När vi kommer till den sista finns det ingen hund att hämta rasen ifrån.

```
for (Dog dog: allTheDogs){  
    System.out.println(dog.getBreed());  
}
```

```
Exception in thread "main" java.lang.NullPointerException: Cannot invoke  
"F5_Mer_om_objekt.Dog.getBreed()" because "dog" is null  
    at F5_Mer_om_objekt.F5.main(F5.java:18)
```

Kolla om null

```
for (Dog dog: allTheDogs){  
    if (dog != null) {  
        System.out.println(dog.getBreed());  
    }  
}
```


Skicka objekt mellan metoder

- Det är möjligt att skicka objekt mellan metoder och klasser i ett program
- Objekt skickas som *referenser*
- Det betyder att om du ändrar på objektet du tar emot ändrar du på objektet du skickade
 - Jämför med att skicka in listor till funktioner i Python

Litet exempel

```
...  
    System.out.println(allTheDogs[0].getName());  
    foo(allTheDogs[0]);  
    System.out.println(allTheDogs[0].getName());  
}  
public void foo(Dog dog){  
    dog.setName("Testing");  
}
```

Unknown

Testing



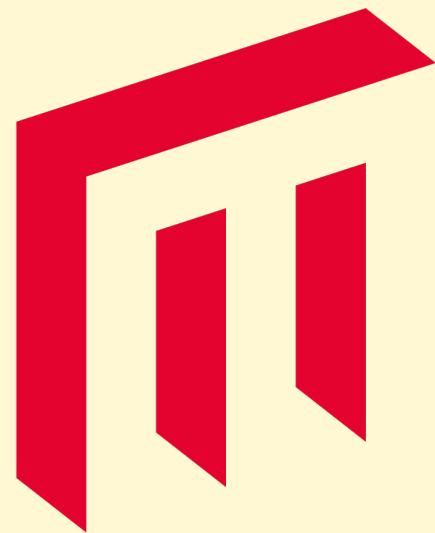
Sammanfattning

Sammanfattning

- Enums kan vara smidiga som konstanter
- Enums kan bli som små mini-objekt
- JavaDoc dokumenterar vår kod genom kommentarer
- Kan översättas till HTML
- toString() är en metod som gör utskrifter snygga
- Objekt kan vara i arrayer
- Vi behöver hantera null
- Enhanced for-loopar är coola
- Vi kan skicka objekt mellan metoder och klasser

Rekommenderad läsning

- Deitel & Deitel, kapitel 8
- Nick Perumov, *Diamantsvärdet och träsvärdet* (Hela serien heter *Svärdens väktare*)



**MALMÖ
UNIVERSITET**

